PROJECT 10 · 23AI2012 FOUNDATIONS OF GANs

The Melody That Changes Its Mood

One LSTM, one opening motif, three temperatures: hear how a single knob changes a tune's character.



Lumi

the LSTM

🚀 Your mission

Train an **LSTM next-note model** on short symbolic melodies. Give it the same four-note opening and let it continue at **three sampling temperatures** (0.5, 1.0, 1.5). Turn each melody into a **five-second WAV**, compare them in a plot and a CSV, and explain the differences with probability rather than with feelings.

👛 What you will make

- a trained LSTM (melody_lstm.pt)
- a training history (cross-entropy and perplexity)
- three 5-second WAVs
- a comparison plot and CSV
- a run_metadata.json provenance record

⚖️ Keep it honest

This is **LSTM symbolic melody generation with temperature-controlled sampling, not a GAN**. There are no emotion labels, so "mood" is a listening impression, not something the model predicts. The default melodies are procedural teaching data, and the audio is a simple sine synthesiser.

🗺️ Your level map

Start with how melodies become tokens, then go inside the LSTM, then see how it learns with cross-entropy. The second half is all about sampling: logits, softmax, temperature, and a careful experiment measured over many seeds.

📝 **Writing your report?** Use `Report_Template.docx` in this kit. Each chapter's grey guidance text tells you which results, figures and files to bring from this booklet.

1 A model that remembers
2 Melodies as tokens
3 Where the training melodies come from
4 Training on shifted sequences
5 Inside the LSTM
6 Learning with cross-entropy
7 From logits to choices: temperature
8 The controlled experiment
9 Making it audible
10 Lab time
11 Three experiments
12 Troubleshooting and cheat sheet

How to read the boxes

- 💡 **Big idea** the one thing to remember
- 🧮 **Work it out** numbers worked out step by step
- 🎯 **Checkpoint** test yourself (answers at the back)
- 🏆 **Boss level** an optional stretch challenge
- 🧪 **Try it yourself** a quick hands-on task
- 🔍 **Code peek** the real lines from src/project.py
- 🐣 **Myth buster** a common mistake, busted
- ⚖️ **Keep it honest** what you may and may not claim

⚡ Quick start

1. Pipeline check

```
python src/project.py --mode smoke-test
```

2. Train (procedural melodies)

```
python src/project.py --mode train --epochs 12 --examples 400 --output-dir outputs/training
```

3. Compare three temperatures

```
python src/project.py --mode generate --checkpoint outputs/training/melody_lstm.pt --temperatures 0.5,1.0,1.5 --seed 7 --output-dir outputs/comparison
```

📖 Words to know

Token
One symbol in a sequence; here, one note (or a start/end marker).

Next-token model
A network that predicts a probability for every possible next symbol.

LSTM
Long Short-Term Memory: a recurrent network with gates and a memory cell.

Teacher forcing
Training on the true previous notes, not the model's own guesses.

Perplexity
exp(cross-entropy): roughly "how many choices the model is torn between".

Temperature
Dividing logits by τ before softmax: low τ sharpens, high τ flattens.

LEVEL 1

A model that remembers



Lumi the LSTM

Hi, I'm **Lumi**, an LSTM. I read a melody one note at a time and keep a little notebook (my *memory cell*) of what has happened so far. At every step I answer one question: "what note comes next?" I don't answer with a single note but with a probability for **every** note.

This project leaves GANs behind for a moment. There is no generator and no discriminator, just one network trained to predict the next note. The interesting part comes at the end: **how we sample** from Lumi's probabilities changes the melody's character, even though the model stays exactly the same.

Project architecture • next-note prediction



At each step: token IDs → learned note vectors → recurrent context → token scores → next note

Figure 1. The full architecture: note tokens → embedding → two stacked LSTM layers → linear layer → softmax over 39 tokens.

Learned versus designed

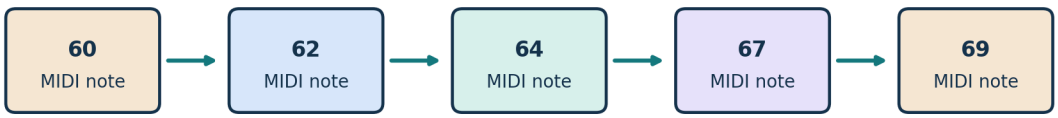
Learned: the next-note probabilities. **Designed:** the opening motif, the temperature, the fixed melody length and the sine synthesiser that turns MIDI numbers into sound.

LEVEL 2

Melodies as tokens

A waveform has tens of thousands of samples per second; a melody, written as **MIDI pitches**, has just a few numbers. MIDI 60 is middle C, 69 is the A above it (440 Hz), and +12 is one octave up.

A melody is a sequence of tokens, not a single continuous sound



Teacher note: the synthesizer later maps MIDI to frequencies; the LSTM predicts symbolic tokens.

Figure 2. A melody is a sequence of tokens, not a single continuous sound.

The vocabulary

Symbol	Internal token	Count
MIDI 48 … 84 (C3 to C6)	MIDI − 48 = 0 … 36	37 pitches
BOS (begin of sequence)	37	1
EOS (end of sequence)	38	1
total		**39**

So MIDI 60 is token 12, and a melody is stored as [BOS, notes…, EOS].

> **"MIDI captures the performance"**
>
> This representation keeps **pitch order only**. It throws away note lengths, rests, loudness, articulation and timbre. Every generated note will simply last the same time. Mention this limitation in your report.

LEVEL 3

Where the training melodies come from

`toy_melodies()` writes procedural tunes, 20 notes each (400 by default, seed 42):

1. pick a **tonic** from MIDI {48, 50, 53, 55, 57, 60};
2. pick one of three five-note **scales**: major pentatonic (0, 2, 4, 7, 9), minor pentatonic (0, 3, 5, 7, 10) or (0, 2, 5, 7, 9);
3. build a **pitch pool**: that scale over three octaves, keeping only notes inside 48…84;
4. start at a random position in the pool and **walk**: each step moves −2, −1, 0, +1 or +2 *positions* with probabilities 0.10, 0.27, 0.26, 0.27, 0.10 (clipped at the ends).

So the melodies are smooth walks along a scale: 26% of moves are "stay", steps that bump into either end of the pool also repeat the note, and big leaps are rare. Measured over the 400 training tunes, consecutive notes differ by **2.0 semitones** on average and **32%** of steps repeat the previous note.

Your own melodies

Instead, supply a CSV with columns `melody_id, step, midi` (MIDI 48–84), and pass `--dataset data/melodies.csv`. Rows are grouped by `melody_id` and sorted by `step`; melodies with fewer than three notes are skipped.

LEVEL 4

Training on shifted sequences

A next-token model learns from pairs: **input** = the sequence without its last symbol, **target** = the sequence without its first symbol.

Position	1	2	3	4
input	BOS	60	62	64
target	60	62	64	EOS

During training Lumi always receives the **true** previous notes, even if she predicted something else. That is **teacher forcing**: it is stable and lets the loss be computed at every position at once. During generation, her own sampled note is fed back in instead.

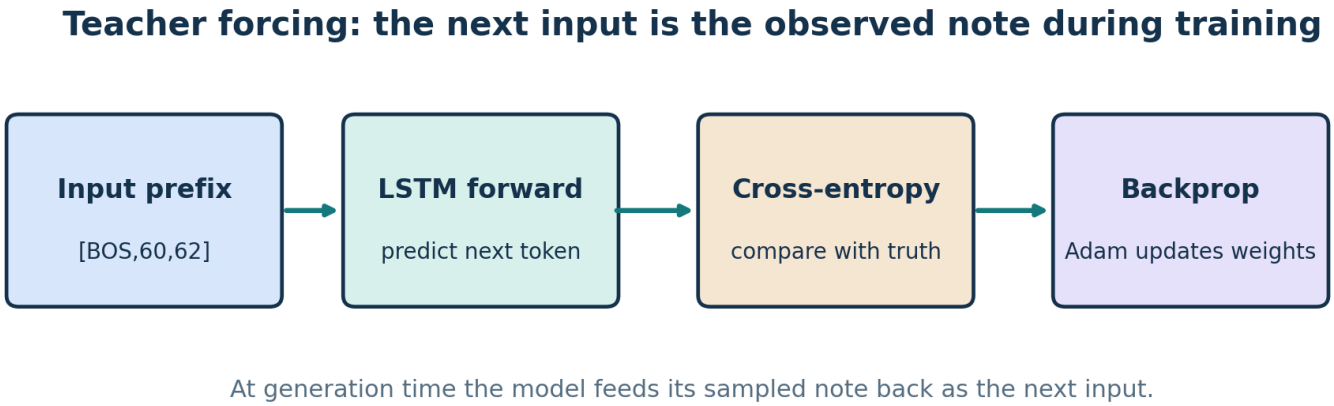


Figure 3. Teacher forcing: during training the next input is the observed note; during generation it is the sampled note.

> 🎯 **Can Lumi cheat?**
>
> At position 3 the input is 62 and the target is 64. Can the network see the 64 while predicting it?
>
> *Answer on the answers page (#1).*

LEVEL 5

Inside the LSTM

Embedding: notes become vectors

A token number like 12 has no useful geometry, since 12 is not "close" to 13 in any musical sense the network can use. `nn.Embedding(39, 32)` gives each token a learnable 32-number vector. Notes that behave alike can end up with similar vectors, but only if training discovers that.

Tensor	Shape (batch 32, 21 steps)
token IDs	(32, 21)
embeddings	(32, 21, 32)
LSTM outputs	(32, 21, 128)
logits	(32, 21, 39)

(21 steps, because each training melody is BOS + 20 notes + EOS = 22 tokens, shifted by one.)

The cell and its gates

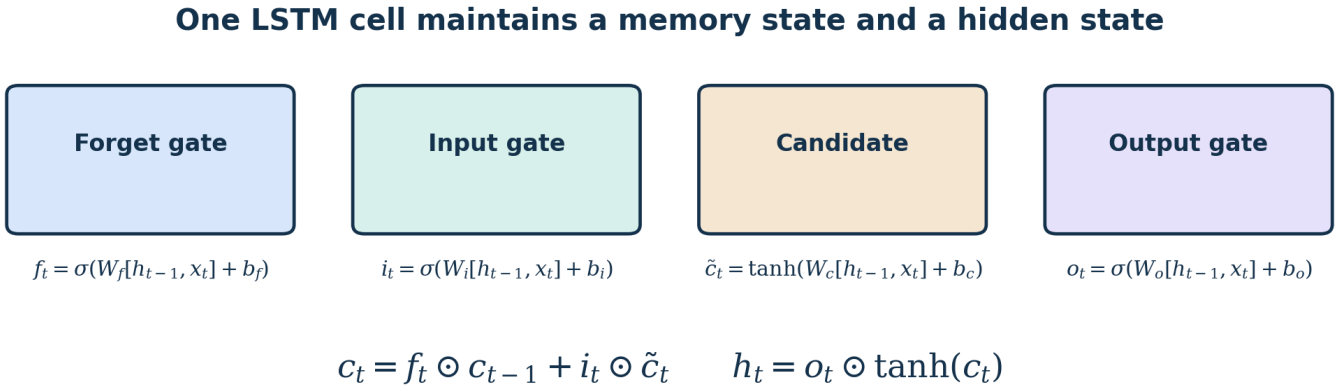


Figure 4. One LSTM cell keeps a memory state c_t and a hidden state h_t, controlled by four learned pathways.

At every step, with input x_t and the previous hidden state h_{t-1}:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \qquad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \qquad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t)$$

- the **forget gate** f_t decides how much old memory to keep;
- the **input gate** i_t decides how much of the new candidate $\tilde{c}_t$ to write;
- the **output gate** o_t decides how much memory to show as h_t.

$\odot$ means element-wise multiplication. Sigmoids give values in (0, 1), like dimmer switches; tanh gives values in $(-1, 1)$.



Lumi the LSTM

That's why I can remember "we're in a minor scale and drifting upwards" for many steps: my forget gate can stay close to 1, so the memory survives. A plain RNN rewrites its whole state at every step and forgets quickly.

The exact model

Layer	Code	Parameters
embedding	`nn.Embedding(39, 32)`	1,248
2 stacked LSTM layers	`nn.LSTM(32, 128, num_layers=2, batch_first=True)`	215,040
output head	`nn.Linear(128, 39)`	5,031
total		**221,319**

The head outputs **39 logits**: unnormalised scores, one per token, turned into probabilities by a softmax.

LEVEL 6

Learning with cross-entropy

If the model gives the correct next token probability p, the loss for that position is $-\log p$. Averaged over all valid positions:

$$L = -\frac{1}{T}\sum_{t=1}^{T} \log p\big(y_t \mid y_1, \ldots, y_{t-1}\big)$$

`nn.CrossEntropyLoss` computes this straight from logits and ignores padded positions marked −100.

How much does a wrong guess cost?

The model predicts three notes with probabilities [0.70, 0.20, 0.10].

Correct note's probability	Loss $-\ln p$
0.70	0.357
0.20	1.609
0.10	2.303

Confidently backing the wrong note is punished hardest.

One update

For each mini-batch of 32 melodies: build padded inputs and targets → forward → loss → `opt.zero_grad()` → `loss.backward()` (backpropagation **through time**, through every step of the unrolled LSTM) → **clip the gradient norm to 1.0** → Adam step (lr 0.002). Clipping matters for recurrent networks: repeated multiplication through many steps can make gradients explode.

Perplexity

$$\text{perplexity} = e^{L}$$

It reads as "the model is about as unsure as if it were choosing uniformly among this many tokens".

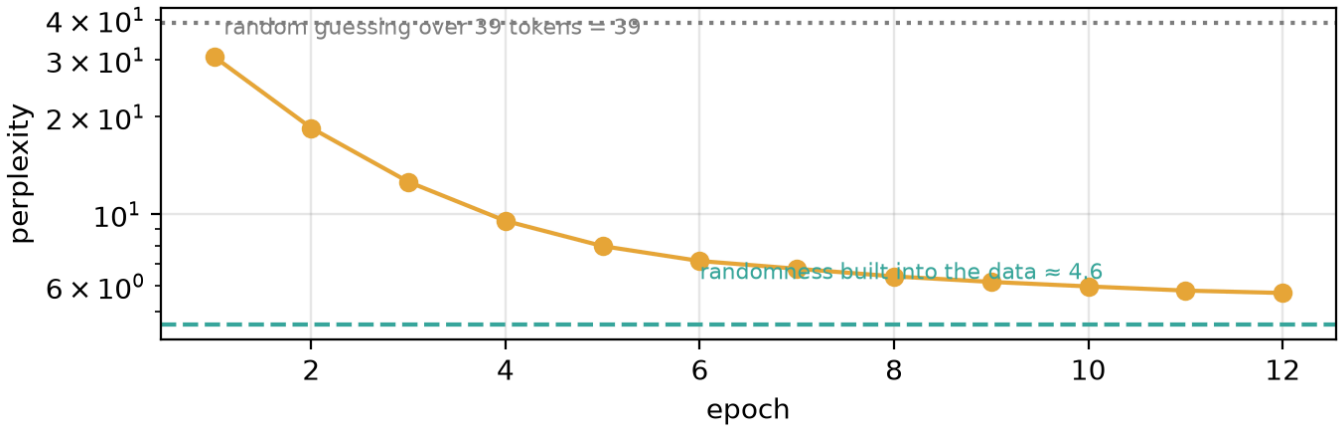


Figure 5. Perplexity from a real 12-epoch run with the defaults (400 melodies). It starts near 31 and ends at 5.70.

How low could it ever go?

The training melodies are themselves random: each move is drawn from probabilities [0.10, 0.27, 0.26, 0.27, 0.10]. The entropy of that choice is $H = -\sum p \ln p = 1.518$, so even a perfect model would have perplexity about $e^{1.518} \approx 4.6$ on those moves. The real run reaches **5.70**, close to that floor (the first note, the scale choice and the range edges add a little extra uncertainty).

"Lower perplexity = better music"

Perplexity says how well the model predicts *this* data. A model can be very predictable and still boring, or varied and incoherent. Always listen and look at the notes too.

LEVEL 7

From logits to choices: temperature

Plain softmax turns logits z into probabilities. Sampling divides them by a **temperature** $\tau > 0$ first:

$$p_i(\tau) = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$$

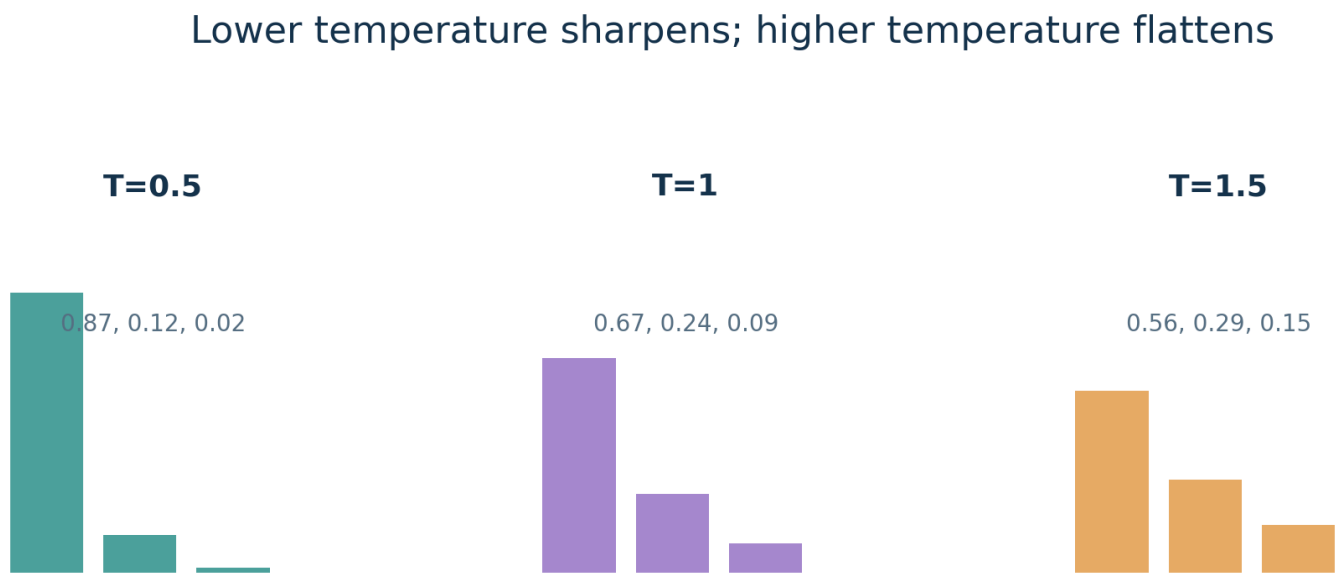


Figure 6. Illustrative logits [2, 1, 0] at three temperatures (not outputs of the trained LSTM).

Derive the three distributions

Logits [2, 1, 0]:

τ	probabilities
0.5	[0.867, 0.117, 0.016]
1.0	[0.665, 0.245, 0.090]
1.5	[0.563, 0.289, 0.148]

The **ranking never changes**; only how strongly the favourite wins. As $\tau \to 0$ sampling becomes "always pick the top note"; as τ grows it approaches a uniform choice.

🎯 **Two notes, two temperatures**

Only two notes are possible, with logits [3, 1]. What are their probabilities at τ = 0.5 and at τ = 2?

Answer on the answers page (#2).

LEVEL 8

The controlled experiment

`sample()` runs the experiment fairly:

- the **same checkpoint** for all three runs;
- the **same opening motif** (60, 62, 64, 67), i.e. C–D–E–G, fed in after BOS to set Lumi's memory;
- the **same random seed** for the sampler (a dedicated `torch.Generator`);
- a fixed **16-note** result: 4 given + 12 sampled;
- BOS and EOS are **disallowed** while sampling, so every step yields an audible note.

🔍 **From sample() in src/project.py**

```
for _ in range(length - len(prefix)):
    logits, state = model(torch.tensor([[current]], device=device), state)
    probs = torch.softmax(logits[0, -1, :len(PITCHES)] / temperature, dim=0)  # 37 pitches only
    nxt = int(torch.multinomial(probs, 1, generator=rng).item())
    current = nxt; notes.append(nxt + 48)
```

One real comparison (seed 7)

τ	16 notes (MIDI)
0.5	60 62 64 67 · 64 62 62 60 60 60 60 62 62 62 60 60
1.0	60 62 64 67 · 64 62 62 60 60 63 60 62 62 62 60 60
1.5	60 62 64 67 · 64 62 58 60 60 63 60 62 63 58 53 68

The three start identically because the seed and the first random draws are shared, and they diverge where a flatter distribution lets a less likely note win.

The same experiment over 50 seeds

One seed can mislead, so we repeated the comparison with seeds 0–49 using the same trained model and averaged:

τ	distinct pitches (of 16)	repeated notes (of 12 sampled steps)	mean jump between notes
0.5	6.5	5.7	1.30 semitones
1.0	7.2	3.7	2.14 semitones

τ	distinct pitches (of 16)	repeated notes (of 12 sampled steps)	mean jump between notes
1.5	8.6	2.5	3.17 semitones
training data	–	*3.8 (32%)*	*2.0 semitones*

At $\tau = 1$ the samples reproduce the training statistics remarkably well, which is good evidence that the model learned the data's habits. Lower temperature repeats more and moves less; higher temperature leaps around more.

🎯 **Attribute the change**

Your friend changed the temperature **and** the opening motif at the same time, and the melody got jumpier. Can they say temperature caused it?

Answer on the answers page (#3).

⚖️ **Probabilities versus feelings**

"At τ = 1.5 the melody used 8.6 distinct pitches on average, compared with 6.5 at τ = 0.5" is a measurement. "τ = 1.5 sounds more anxious to me" is a listening impression; label it as one. The model has no idea what a mood is.

LEVEL 9

Making it audible

`notes_to_wav()` turns MIDI numbers into a 5-second, 22,050 Hz WAV:

- $f = 440 \cdot 2^{(m-69)/12}$ Hz for each note;
- 16 equal notes in 5 s, so each lasts **0.3125 s**; boundaries are computed on the sample grid so the total is exact;
- each note is $\sin(2\pi f t) + 0.18\sin(4\pi f t)$, with an 18 ms attack and a 65 ms release to avoid clicks;
- the whole clip is normalised so its peak is 0.7.

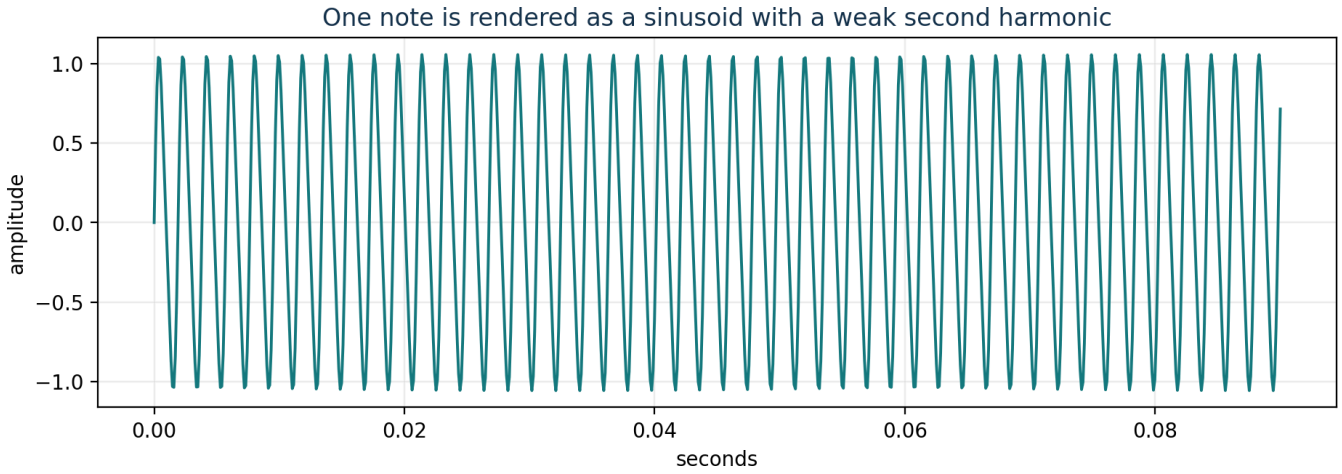


Figure 7. One note: a sinusoid with a weak second harmonic.

The outputs are `temperature_0p5.wav`, `temperature_1.wav`, `temperature_1p5.wav`, `melody_comparison.png` (three coloured piano-roll strips, labelled with the real note count and duration), `melody_comparison.csv` (temperature, step, MIDI, frequency) and `run_metadata.json`.

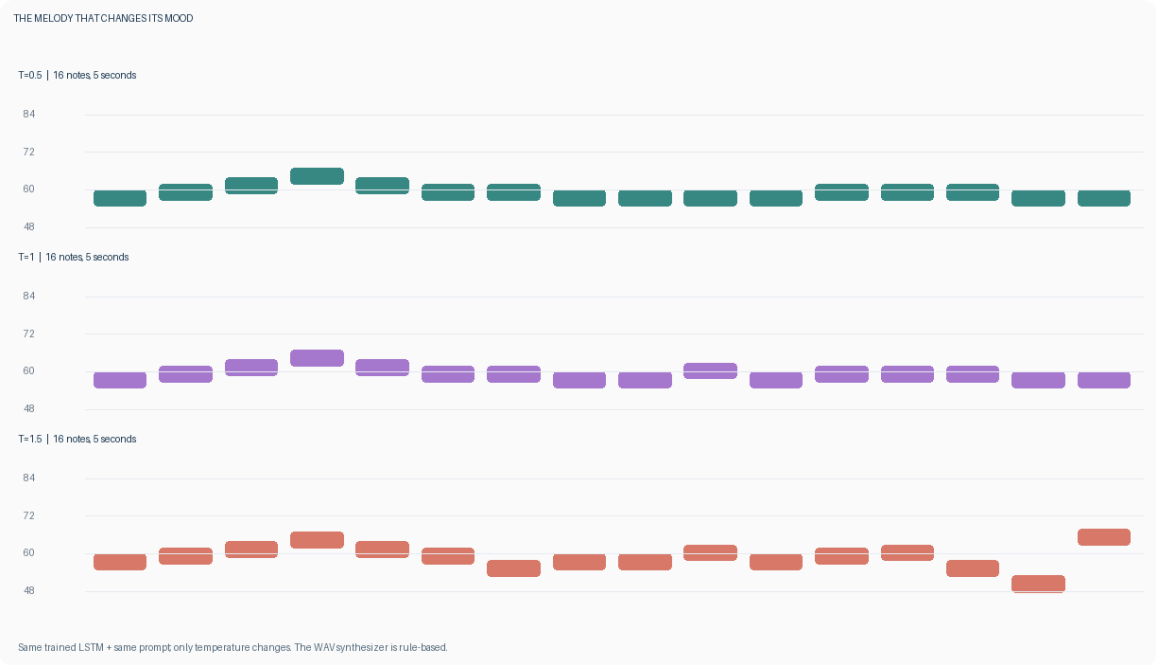


Figure 8. The real comparison plot from the seed-7 run.

LEVEL 10

Lab time

```
python src/project.py --mode smoke-test
python src/project.py --mode train --epochs 12 --examples 400 --output-dir outputs/training
python src/project.py --mode generate --checkpoint outputs/training/melody_lstm.pt --temperatures
0.5,1.0,1.5 --seed 7 --output-dir outputs/comparison
```

The **smoke test** trains one epoch on 32 melodies and runs the three-temperature export into `outputs/smoke_test`. With a 1-epoch model all three melodies may come out the same, because its probabilities are nearly flat and temperature has little to sharpen. The older style (`python src/project.py train ...`) still works.

Option	Default	Meaning
`--examples`, `--epochs`	400, 12	data size and training length
`--batch-size`, `--lr`	32, 0.002	Adam settings
`--embedding`, `--hidden`, `--layers`	32, 128, 2	model size
`--temperatures`	0.5,1.0,1.5	comma-separated list
`--seed` (generate)	7	sampler seed
`--length`, `--seconds`	16, 5	notes and WAV duration

LEVEL 11

Three experiments

Experiment	Change only	Measure
A. Temperature	τ = 0.5 / 1.0 / 1.5	distinct pitches, repeats, jumps (several seeds)
B. Opening motif	`prefix` in `sample()`	does the model keep the motif's shape and scale?
C. Training length	1 vs 4 vs 12 epochs	perplexity versus how coherent it sounds

Find the "data temperature"

Write a loop that tries τ from 0.6 to 1.4 in steps of 0.1, samples 50 melodies at each, and finds the τ whose average jump size and repeat rate are closest to the training data's (2.0 semitones, 32% repeats). Is it exactly 1.0? Explain why it should be close.

LEVEL 12

Troubleshooting and cheat sheet

Symptom	Likely cause	Check
all three clips identical	under-trained, very flat probabilities	train longer; look at the perplexity
chaotic melodies at every τ	bad checkpoint or token range	MIDI inside 48–84; retrain
`MIDI … outside supported 48..84`	CSV pitch out of range	transpose your melodies
`temperature must be > 0`	τ = 0 given	use a small value such as 0.1

Item	Value
Tokens	37 pitches (MIDI 48–84) + BOS + EOS = 39
Data	400 procedural 20-note scale walks (3 pentatonic scales)
Model	Embedding(39, 32) → 2 × LSTM(128) → Linear(128, 39); 221,319 parameters
Training	cross-entropy, Adam lr 0.002, batch 32, grad-norm clip 1.0, 12 epochs
Sampling	prefix 60-62-64-67, 12 sampled notes, same seed for each τ
Audio	16 × 0.3125 s, sine + 0.18 × 2nd harmonic, 22,050 Hz, peak 0.7



Lumi the LSTM

Your viva in one breath: I'm an LSTM next-note model trained with cross-entropy and teacher forcing; temperature rescales my logits before sampling; the experiment keeps everything fixed except τ; and I'm not a GAN.

References: Hochreiter & Schmidhuber (1997) *Long Short-Term Memory*; the PyTorch documentation for `nn.LSTM` and `nn.CrossEntropyLoss`; Valerio Velardo's *Generating Melodies with a Neural Network* lessons (a conceptual companion, not this code).

✅ Answers to the checkpoints

#1 · Can Lumi cheat?

No. An LSTM processes the sequence left to right, so at position 3 its state contains only BOS, 60 and 62. The 64 appears only as the target for the loss.

#2 · Two notes, two temperatures

$\tau = 0.5$: logits become [6, 2], so $p = [e^6, e^2]/(e^6 + e^2) = [0.982, 0.018]$. $\tau = 2$: logits become [1.5, 0.5], so $p = [0.731, 0.269]$. The low temperature almost always picks the first note; the high one gives the second note a real chance.

#3 · Attribute the change

No. Two variables changed, so the effect can't be attributed to either one. Keep the checkpoint, motif and seed fixed and change only τ.